LAPORAN PRAKTIKUM

IOT 2

Modul 6: Protokol Komunikasi Telemetri Industri (MQTT)

Oleh :

Nama : Mita Fitri Nadori

NPM : 25782102

Kelas : TRI (3D)

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

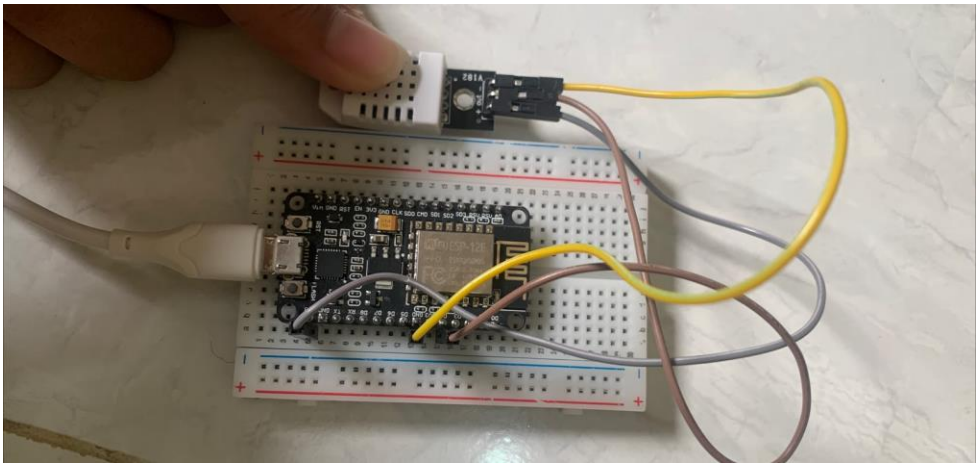
POLITEKNIK NEGERI LAMPUNG

2026/2027

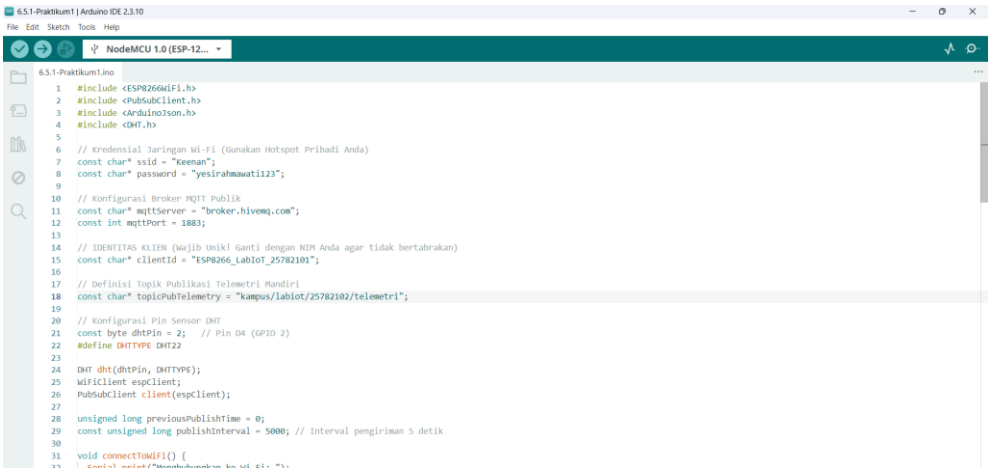
6.5 Praktikum

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Pada rangkaian ini, NodeMCU ESP8266 dihubungkan dengan sensor DHT22 menggunakan kabel jumper. Rangkaian ini digunakan untuk membaca suhu dan kelembapan di sekitar sensor.



Kode program ini dibuat untuk menghubungkan NodeMCU ke Wi-Fi dan broker MQTT. Data suhu dan kelembapan dari sensor kemudian dikirim dalam bentuk JSON.



```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <DHT.h>

// Kredensial Jaringan Wi-Fi (Gunakan Hotspot Pribadi Anda)
const char* ssid = "Keenan";
const char* password = "yesirahmawati123";

// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;

// IDENTITAS KLIEN (Wajib Unik! Ganti dengan NIM Anda agar tidak bertabrakan)
const char* clientId = "ESP8266_LabIoT_25782101";

// Definisi Topik Publikasi Telemetri Mandiri
const char* topicPubTelemetry = "kampus/labiot/25782102/telemetri";

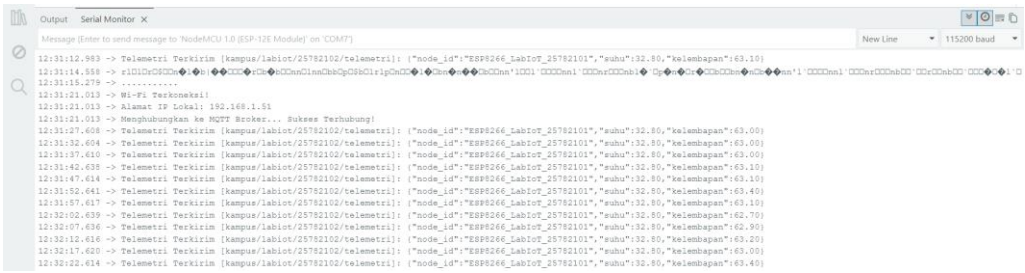
// Konfigurasi Pin Sensor DHT
const byte dhtPin = 2;   // Pin D4 (GPIO 2)
#define DHTTYPE DHT22

DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);

unsigned long previousPublishTime = 0;
const unsigned long publishInterval = 5000; // Interval pengiriman 5 detik

void connectToWiFi() {
```

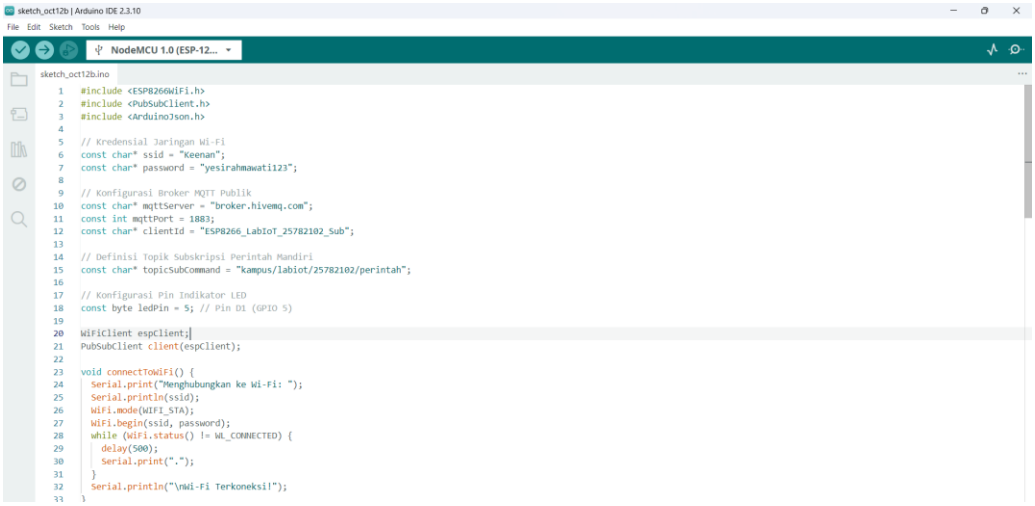
Pada Serial Monitor terlihat data sensor yang dikirim oleh NodeMCU secara berkala. Hasil ini menunjukkan bahwa proses pengiriman data telemetri sudah berjalan.



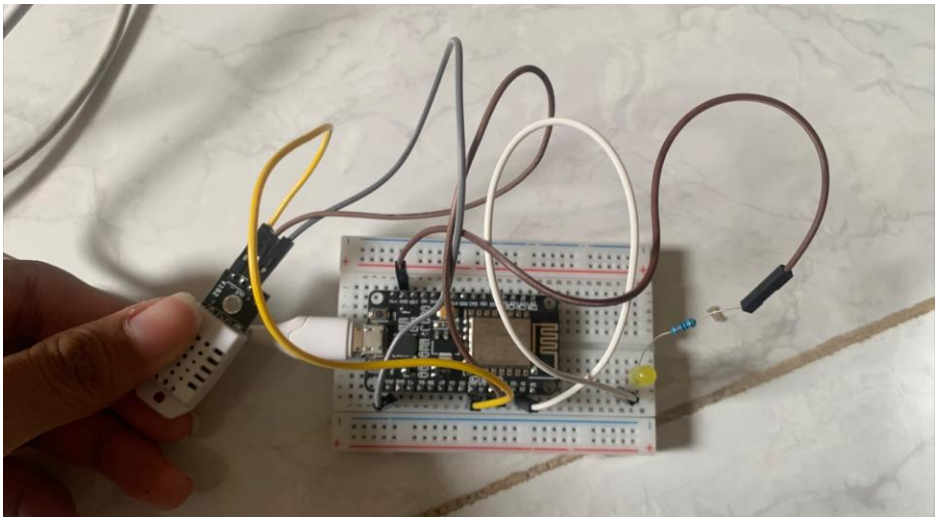
```
Output  Serial Monitor
12:31:12.983 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.10}
12:31:14.558 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.10}
12:31:15.279 -> ..........
12:31:21.013 -> Wi-Fi Terkoneksi!
12:31:21.013 -> Alamat IP Lokal: 192.168.1.51
12:31:21.013 -> Menghubungkan ke MQTT Broker... Sukses Terhubung!
12:31:27.608 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.00}
12:31:32.604 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.00}
12:31:37.610 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.00}
12:31:42.638 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.10}
12:31:47.614 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.10}
12:31:52.641 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.40}
12:31:57.617 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.10}
12:32:02.639 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":62.70}
12:32:07.636 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":62.90}
12:32:12.616 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.20}
12:32:17.620 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.00}
12:32:22.614 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782101","suhu":32.80,"kelembapan":63.40}
```

6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

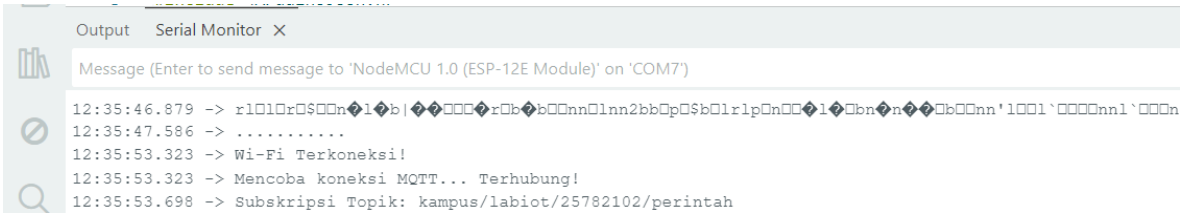
Kode ini digunakan untuk menerima pesan dari broker MQTT melalui topik kendali. Pesan yang diterima akan diproses untuk mengatur kondisi LED.



Rangkaian ini menggunakan NodeMCU, sensor DHT22, dan LED yang dipasang pada breadboard. LED digunakan sebagai indikator untuk mencoba perintah yang dikirim melalui MQTT.

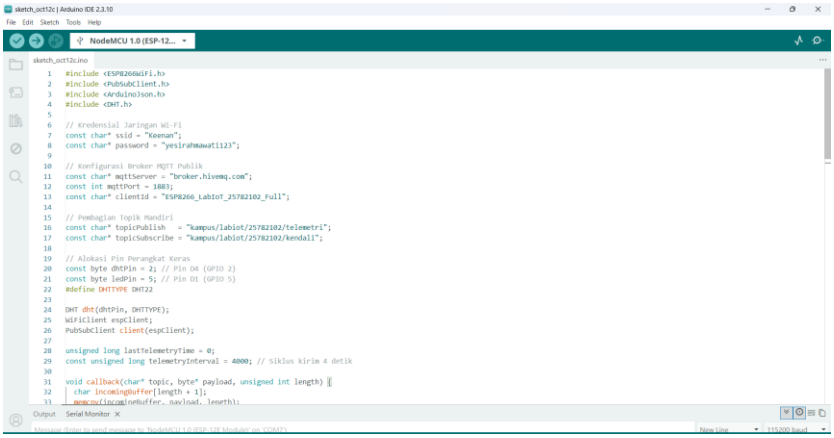


Pada Serial Monitor terlihat proses penyambungan NodeMCU ke Wi-Fi dan broker MQTT. Setelah terhubung, perangkat dapat menerima perintah melalui topik yang sudah ditentukan.

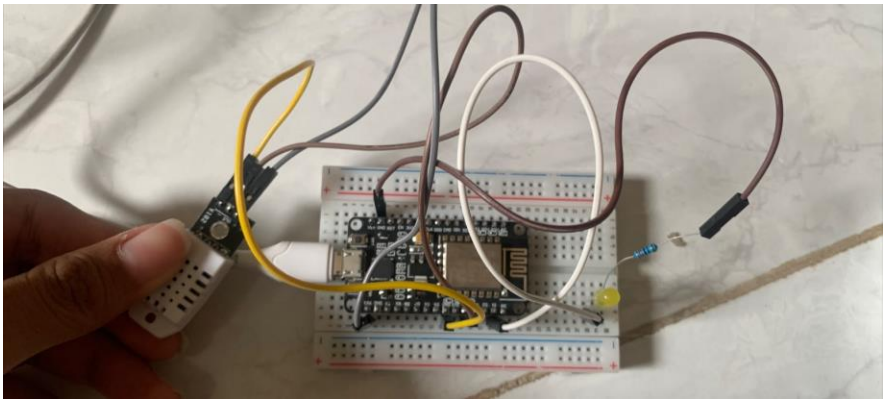


6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

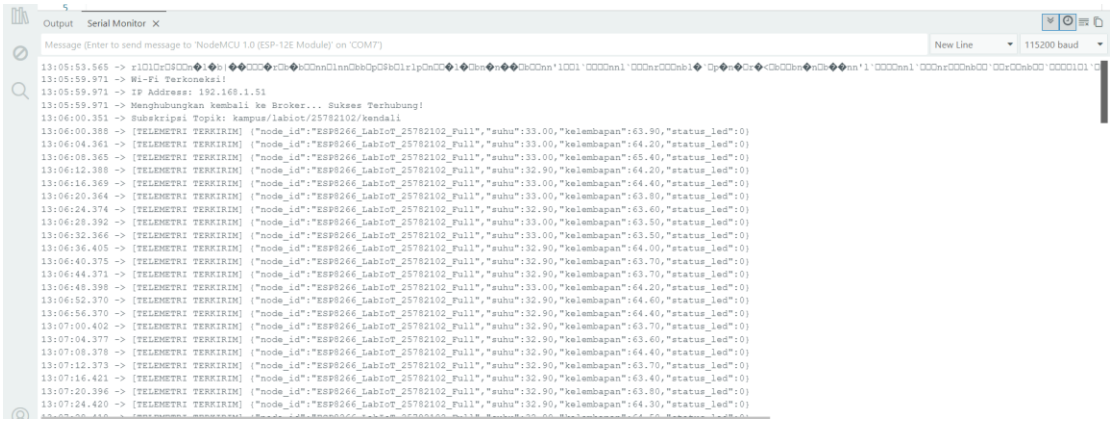
Kode program ini menggabungkan pembacaan sensor dan kontrol LED dalam satu sistem. Data sensor dikirim setiap empat detik, sedangkan LED dapat dikendalikan melalui pesan MQTT.



Rangkaian NodeMCU dan sensor DHT22 digunakan untuk menjalankan sistem telemetri dan kontrol dua arah. LED juga dipasang sebagai indikator yang dapat dikendalikan melalui MQTT.



Serial Monitor menampilkan proses koneksi dan pengiriman data sensor dari NodeMCU. Data tersebut berisi informasi suhu, kelembapan, dan status LED.

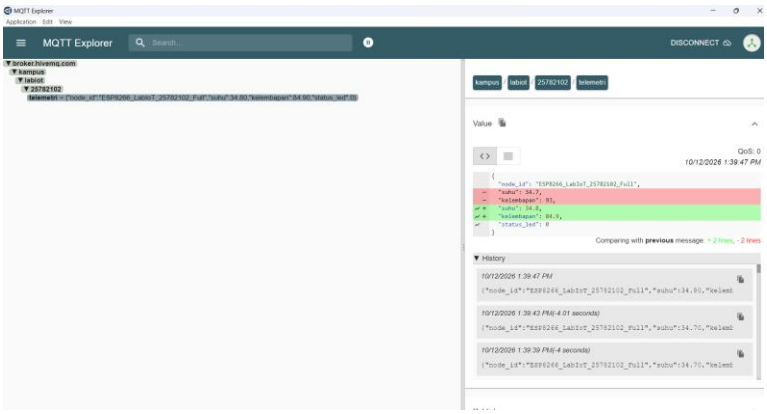


6.5.4 Panduan Pengujian Diagnostik dengan MQTT Explorer

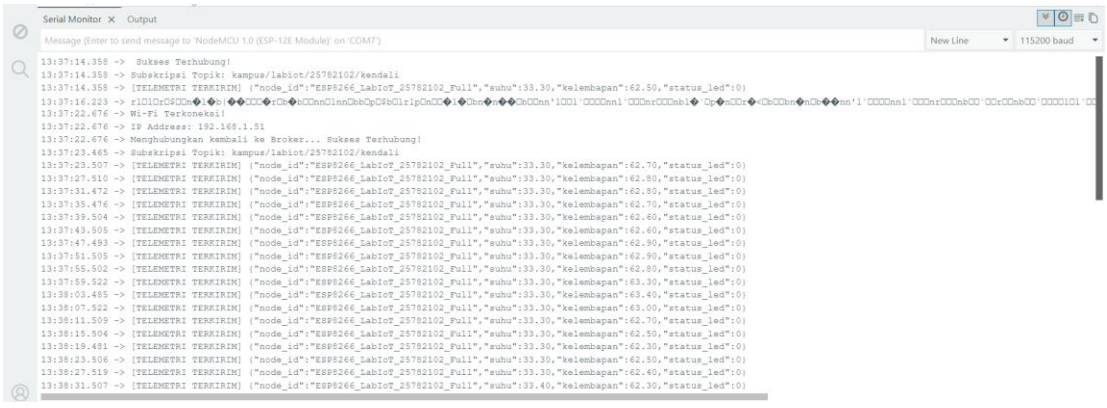
Program ini digunakan untuk mengirim data sensor ke broker dan menerima perintah dari MQTT Explorer. Dengan program ini, pengiriman data dan kontrol LED dapat dilakukan secara bersamaan.



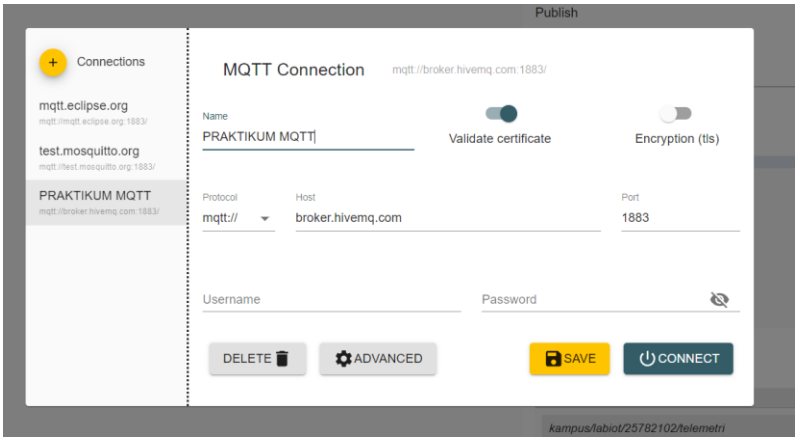
MQTT Explorer digunakan untuk melihat pesan yang dikirim oleh NodeMCU melalui topik telemetri. Data yang masuk dapat dipantau untuk mengetahui apakah komunikasi MQTT berjalan dengan baik.



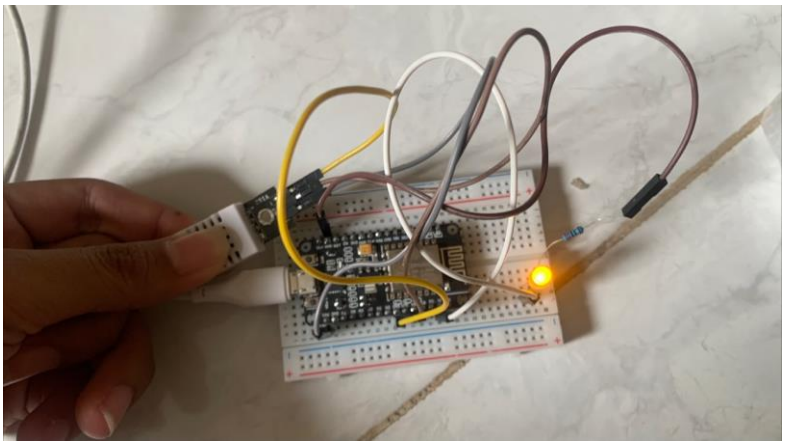
Pada gambar ini terlihat hasil komunikasi perangkat selama program dijalankan. Serial Monitor membantu melihat data yang dikirim serta mengetahui kondisi koneksi MQTT.



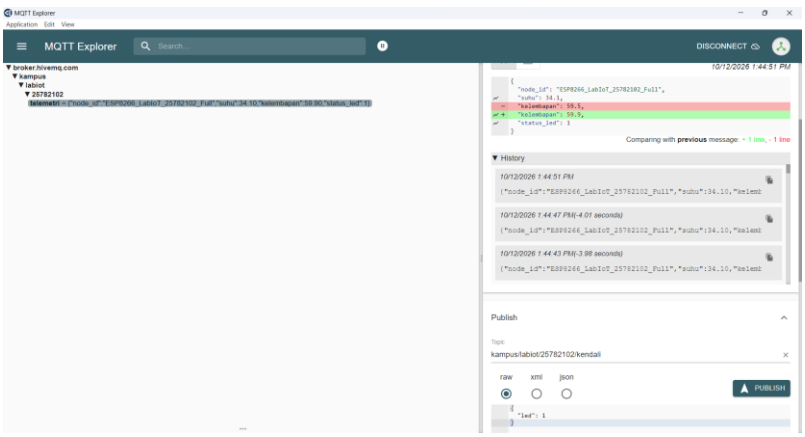
Pada bagian ini, koneksi MQTT diatur menggunakan broker broker.hivemq.com dan port 1883. Pengaturan ini digunakan agar MQTT Explorer dapat terhubung ke broker yang sama dengan NodeMCU.



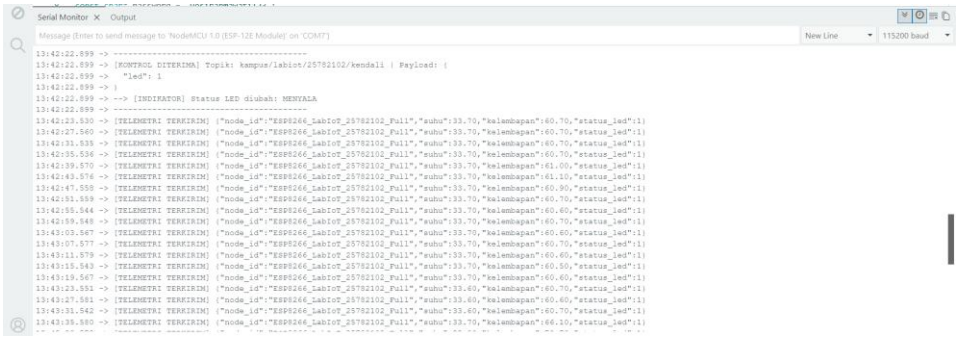
LED pada rangkaian digunakan untuk menguji perintah yang dikirim melalui MQTT Explorer. Ketika perintah menyalakan LED dikirim, lampu indikator terlihat menyala.



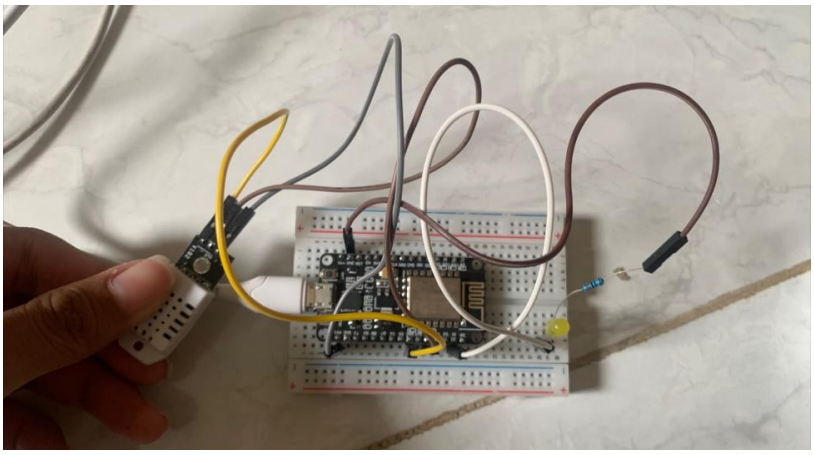
Pada MQTT Explorer terlihat topik yang digunakan untuk menerima data dari NodeMCU. Pesan yang masuk dapat diperiksa untuk melihat perubahan nilai telemetri selama pengujian.



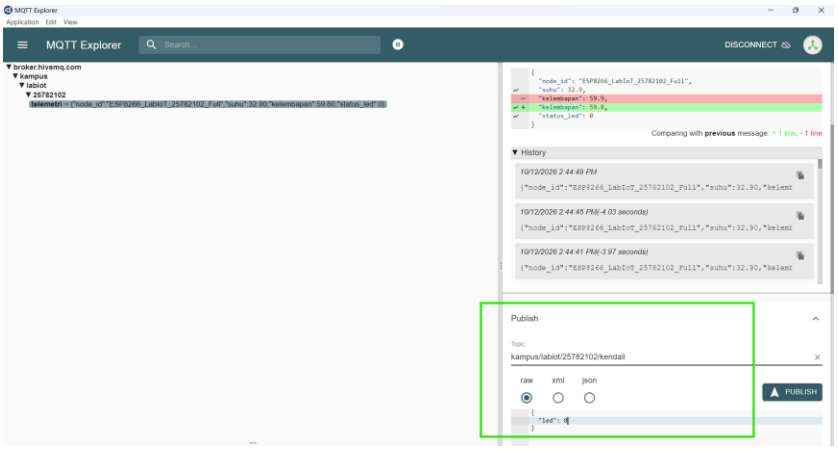
Serial Monitor menampilkan data suhu dan kelembapan yang dikirim secara berkala ke broker MQTT. Hasil ini digunakan untuk memastikan bahwa perangkat tetap mengirim data selama pengujian.



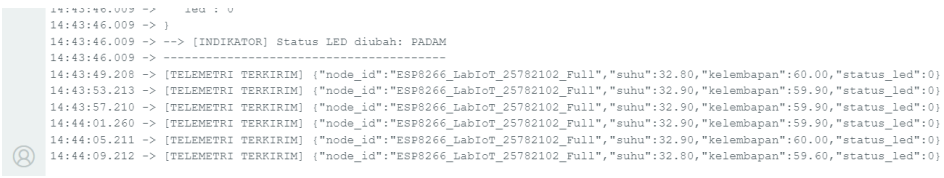
Rangkaian ini terdiri dari NodeMCU ESP8266 dan sensor DHT22 yang dihubungkan menggunakan kabel jumper. Perangkat digunakan untuk melakukan pengujian pembacaan sensor dan komunikasi MQTT.



Pada kolom Publish, topik kendali dan payload JSON dimasukkan untuk mengirim perintah ke NodeMCU. Perintah tersebut digunakan untuk menyalakan atau mematikan LED sesuai nilai yang dikirim.



Pada Serial Monitor terlihat hasil pengiriman telemetri setelah pengujian perintah melalui MQTT Explorer. Hasil ini digunakan untuk memeriksa komunikasi antara NodeMCU dan aplikasi MQTT Explorer.



6.6 Latihan

Kerjakan latihan berikut secara mandiri berbasis analisis teori dan modifikasi kode program untuk melengkapi laporan praktikum Anda:

- Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):

  Pada Modul 3, kita menggunakan protokol HTTP untuk membaca dan mengontrol perangkat. Jelaskan secara mendalam mengapa protokol MQTT jauh lebih unggul daripada HTTP dalam hal:
  - Efisiensi penggunaan kapasitas bandwidth pada paket data seluler.
  - Beban komputasi mikrokontroler (CPU overhead) saat menangani pertukaran data dua arah.
  - Kemampuan pengiriman data secara simultan ke banyak perangkat penerima (1-to-N distribution).

  Jawab:

  Menurut saya, MQTT lebih efisien dibandingkan HTTP karena ukuran header pesan MQTT lebih kecil sehingga data yang dikirim melalui jaringan tidak terlalu besar. MQTT memiliki header tetap minimal 2 byte, sedangkan HTTP biasanya menggunakan header yang lebih besar karena memuat informasi seperti metode permintaan, alamat, dan berbagai metadata lainnya.

  Selain itu, MQTT menggunakan koneksi yang tetap terbuka antara perangkat dan broker. Jadi, ketika NodeMCU mengirim data suhu dan kelembapan secara berkala, perangkat tidak perlu membuat koneksi baru untuk setiap pengiriman. Hal ini dapat mengurangi penggunaan bandwidth dan membuat komunikasi lebih hemat, terutama jika perangkat menggunakan paket data seluler yang terbatas.

- Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):

  Sebuah pabrik manufaktur memiliki 3 lini produksi (line1, line2, line3), di mana setiap lini memiliki 4 mesin (m1, m2, m3, m4), dan setiap mesin melaporkan metrik suhu, getaran, dan status. Struktur topiknya adalah pabrik/<line>/<mesin>/<sensor>. Tentukan ekspresi topik subskripsi (subscription topic) yang tepat menggunakan karakter wildcard (+ atau #) untuk:
  - Memantau parameter suhu dari seluruh mesin di semua lini produksi.
  - Memantau seluruh parameter sensor (suhu, getaran, status) khusus untuk mesin m1 di lini line2.
  - Memantau seluruh lalu lintas data tanpa terkecuali dari seluruh lini dan seluruh mesin di pabrik tersebut.

  Jawab:

  MQTT dapat mengurangi beban pemrosesan mikrokontroler karena komunikasi dilakukan melalui koneksi yang tetap terhubung dan format pesan yang relatif sederhana. Pada NodeMCU ESP8266, fungsi client.loop() digunakan untuk memproses pesan masuk dan menjaga komunikasi dengan broker, sedangkan fungsi callback digunakan untuk menangani perintah yang diterima.

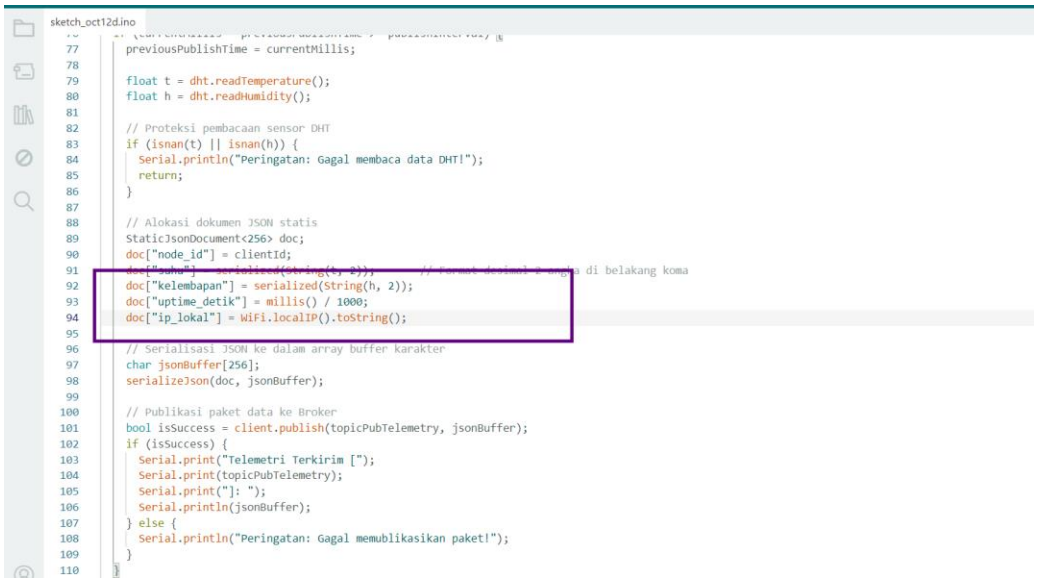
  Berbeda dengan HTTP yang umumnya menggunakan pola request-response, MQTT memungkinkan perangkat mengirim data sensor sekaligus menerima perintah tanpa harus membuat permintaan baru untuk setiap komunikasi. Dengan demikian, program dapat lebih mudah menangani pengiriman data telemetri dan pengendalian LED secara bersamaan. Meskipun demikian, beban CPU sebenarnya tetap bergantung pada program, jumlah pesan, dan proses pengolahan data yang dilakukan.

- Latihan 3 (Implementasi Serialisasi Data Dinamis):

  Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:
  - "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
  - "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()). Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!

Jawab:

Kode yang dimodifikasi:



```
sketch_oct12d.ino
  previousPublishTime = currentMillis;

  float t = dht.readTemperature();
  float h = dht.readHumidity();

  // Proteksi pembacaan sensor DHT
  if (isnan(t) || isnan(h)) {
    Serial.println("Peringatan: Gagal membaca data DHT!");
    return;
  }

  // Alokasi dokumen JSON statis
  StaticJsonDocument<256> doc;
  doc["node_id"] = clientId;
  doc["suhu"] = serialized(String(t, 2));      // Format desimal 2 angka di belakang koma
  doc["kelembapan"] = serialized(String(h, 2));
  doc["uptime_detik"] = millis() / 1000;
  doc["ip_lokal"] = WiFi.localIP().toString();

  // Serialisasi JSON ke dalam array buffer karakter
  char jsonBuffer[256];
  serializeJson(doc, jsonBuffer);

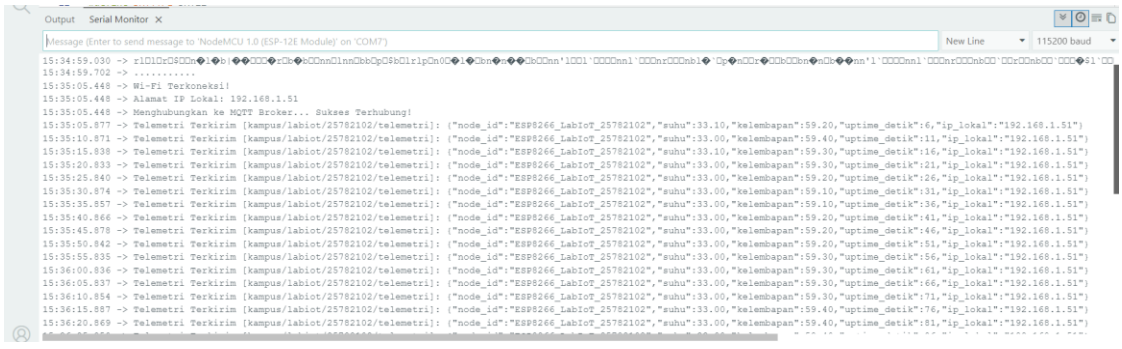
  // Publikasi paket data ke Broker
  bool isSuccess = client.publish(topicPubTelemetry, jsonBuffer);
  if (isSuccess) {
    Serial.print("Telemetri Terkirim [");
    Serial.print(topicPubTelemetry);
    Serial.print("]: ");
    Serial.println(jsonBuffer);
  } else {
    Serial.println("Peringatan: Gagal memublikasikan paket!");
  }
}
```

Penjelasan:

1. doc["uptime_detik"] = millis() / 1000; digunakan untuk menghitung lama waktu perangkat aktif sejak program mulai berjalan. Fungsi millis() menghasilkan waktu dalam milidetik, sehingga nilainya dibagi 1000 agar menjadi detik.
2. doc["ip_lokal"] = WiFi.localIP().toString(); digunakan untuk mengambil alamat IP lokal yang diperoleh NodeMCU setelah terhubung ke Wi-Fi. Fungsi toString() mengubah alamat IP tersebut menjadi teks agar dapat dimasukkan ke dalam dokumen JSON.

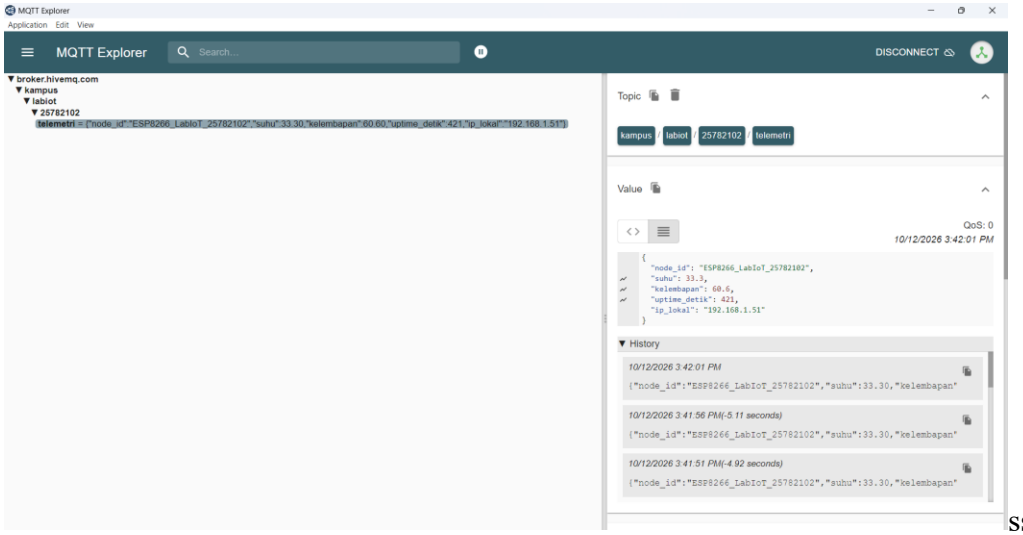
Kedua baris kode tersebut harus ditambahkan sebelum fungsi serializeJson() dipanggil. Dengan demikian, informasi uptime dan alamat IP akan ikut dikirim bersama data suhu dan kelembapan.

Hasil Output:



```
Output  Serial Monitor ×
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM7')        New Line    115200 baud
15:34:59.030 -> rlD1□r□$□□n□1□b!□□□□□r□k□b□□nn□1nn□bb□p□$b□1r1p□n□□□1□□bn□n□□b□□nn'1□□1'□□□□nn1'□□□nr□□□nb1□'□p□n□□r□□□b□□bn□n□b□□nn'1'□□□□nn1'□□□nr□□□nb□□'□□r□□nb□□'□□□□$1'□□
15:34:59.702 -> .....
15:35:05.448 -> Wi-Fi Terkoneksi!
15:35:05.448 -> Alamat IP Lokal: 192.168.1.51
15:35:05.448 -> Menghubungkan ke MQTT Broker... Sukses Terhubung!
15:35:05.877 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.10,"kelembapan":59.20,"uptime_detik":6,"ip_lokal":"192.168.1.51"}
15:35:10.871 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.40,"uptime_detik":11,"ip_lokal":"192.168.1.51"}
15:35:15.838 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.10,"kelembapan":59.30,"uptime_detik":16,"ip_lokal":"192.168.1.51"}
15:35:20.833 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.30,"uptime_detik":21,"ip_lokal":"192.168.1.51"}
15:35:25.840 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.20,"uptime_detik":26,"ip_lokal":"192.168.1.51"}
15:35:30.874 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.10,"uptime_detik":31,"ip_lokal":"192.168.1.51"}
15:35:35.857 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.10,"uptime_detik":36,"ip_lokal":"192.168.1.51"}
15:35:40.866 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.20,"uptime_detik":41,"ip_lokal":"192.168.1.51"}
15:35:45.878 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.20,"uptime_detik":46,"ip_lokal":"192.168.1.51"}
15:35:50.842 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.20,"uptime_detik":51,"ip_lokal":"192.168.1.51"}
15:35:55.835 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.30,"uptime_detik":56,"ip_lokal":"192.168.1.51"}
15:36:00.836 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.30,"uptime_detik":61,"ip_lokal":"192.168.1.51"}
15:36:05.837 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.30,"uptime_detik":66,"ip_lokal":"192.168.1.51"}
15:36:10.854 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.30,"uptime_detik":71,"ip_lokal":"192.168.1.51"}
15:36:15.887 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.40,"uptime_detik":76,"ip_lokal":"192.168.1.51"}
15:36:20.869 -> Telemetri Terkirim [kampus/labiot/25782102/telemetri]: {"node_id":"ESP8266_LabIoT_25782102","suhu":33.00,"kelembapan":59.40,"uptime_detik":81,"ip_lokal":"192.168.1.51"}
```

Hadil di MQTT:



ss

- Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):

  Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

  - Apa konsekuensi teknis yang terjadi terhadap responsivitas penerimaan perintah callback jika seorang pengembang menyisipkan instruksi delay(6000) di dalam fungsi loop()?

    Jawab:

    Penggunaan delay(6000) membuat program berhenti selama 6 detik. Selama waktu tersebut, fungsi client.loop() tidak berjalan sehingga pesan yang masuk dari broker MQTT tidak bisa langsung diproses oleh callback.

    Akibatnya, respons perangkat terhadap perintah dari MQTT Explorer menjadi terlambat. Jika kondisi ini berlangsung terlalu lama, koneksi MQTT juga bisa terganggu. Oleh karena itu, penggunaan delay() sebaiknya dikurangi agar komunikasi MQTT tetap berjalan dengan baik.

  - Jelaskan bagaimana fungsi reconnect() bekerja memulihkan koneksi saat router Wi-Fi atau broker MQTT sempat terputus sesaat!

    Jawab:

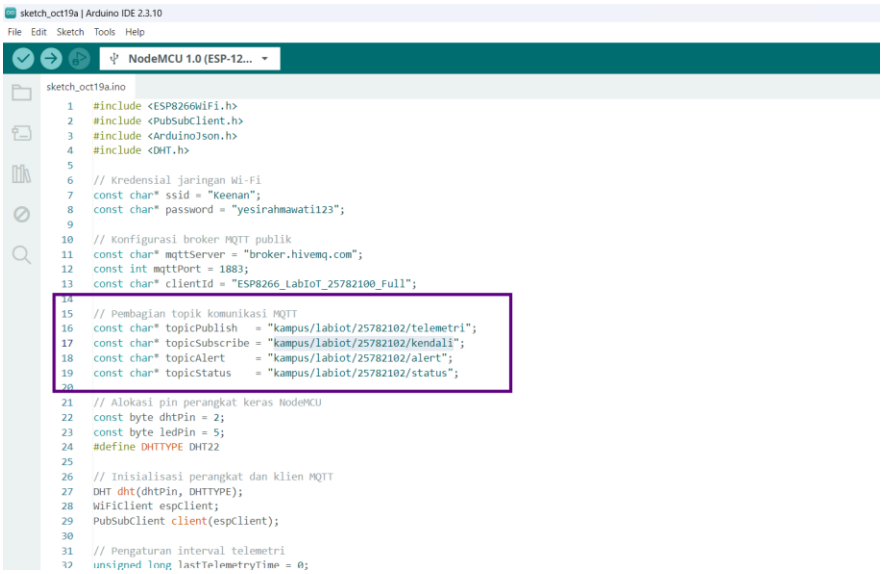
    Fungsi reconnect() digunakan untuk mencoba menghubungkan kembali ESP8266 ke broker MQTT ketika koneksi terputus. Program akan memeriksa status koneksi dan mencoba terhubung kembali sampai berhasil.

Setelah koneksi berhasil, ESP8266 akan berlangganan kembali ke topik kontrol menggunakan client.subscribe(). Dengan begitu, perangkat bisa menerima perintah dari MQTT Explorer lagi tanpa harus menjalankan ulang program secara manual.

6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

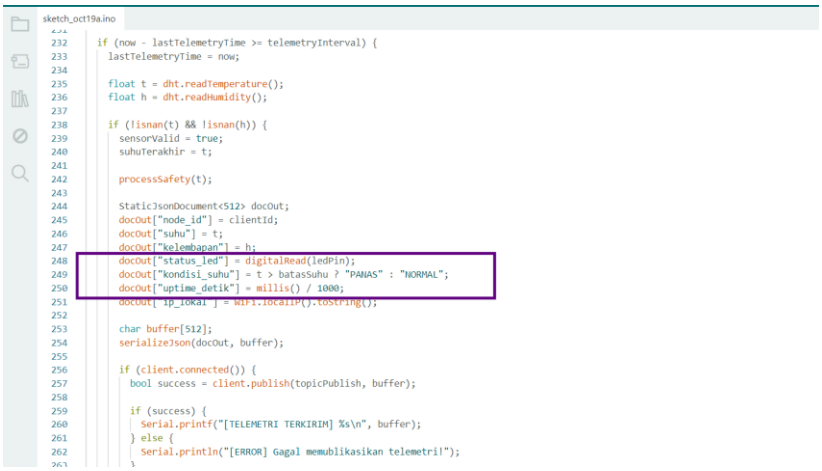
Kode:

1. Konfigurasi topik dan pin mengenalkan perangkat dan jalur komunikasinya



2. Rule engine suhu





3. Alert dan LED otomatis



4. Uji pengamanan kendali saat suhu bahaya



5. LWT



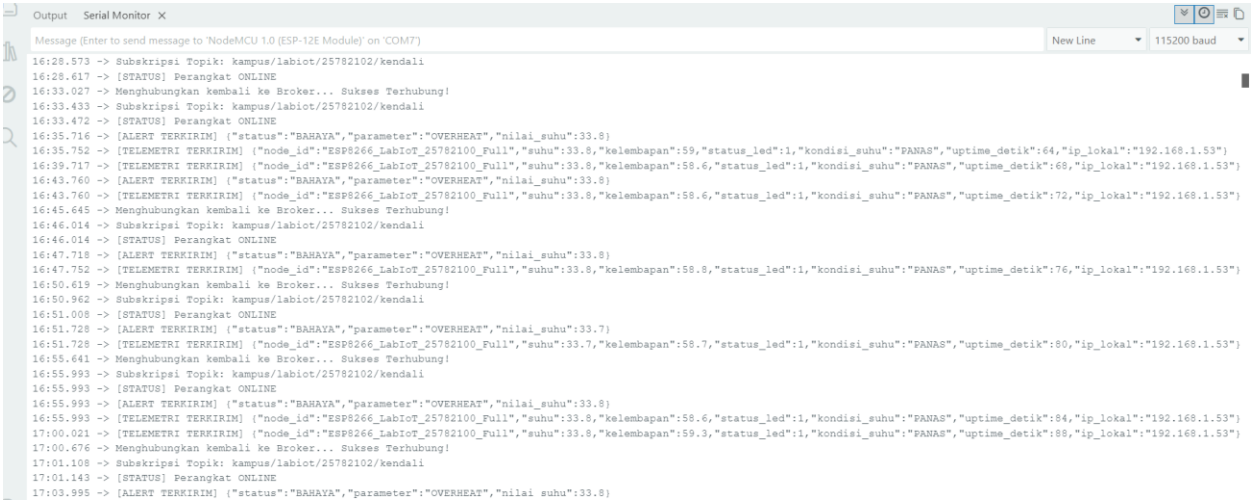
```
sketch_oct19a.ino
48  const float batasSuhu = 31.0;
49
50  // Status keselamatan dan validitas sensor
51  bool dangerActive = false;
52  bool alertLatched = false;
53  bool sensorValid = false;
54  bool wifiConnectedPreviously = false;
55
56  // Penyimpan suhu terakhir
57  float suhuTerakhir = NAN;
58
59  // Pesan Last Will dan status ONLINE
60  const char* willMessage = "{\"status\":\"OFFLINE\",\"keterangan\":\"Perangkat Terputus Tak Terduga\"}";
61  const char* onlineMessage = "{\"status\":\"ONLINE\",\"keterangan\":\"Perangkat Beroperasi Normal\"}";
62
63  // Memproses perintah kendali LED dari MQTT
64  void callback(char* topic, byte* payload, unsigned int length) {
65    char incomingBuffer[length + 1];
66    memcpy(incomingBuffer, payload, length);
67    incomingBuffer[length] = '\0';
68
69    Serial.println("---------------------------------------");
70    Serial.printf("[KONTROL DITERIMA] Topik: %s | Payload: %s\n",
71                  topic, incomingBuffer);
72
73    StaticJsonDocument<256> doc;
74    DeserializationError err = deserializeJson(doc, incomingBuffer);
75
```

6. Uji perangkat tersambung kembali

```
219  void loop() {
220    maintainWiFi();
221
222    if (WiFi.status() == WL_CONNECTED) {
223      if (!client.connected()) {
224        reconnect();
225      }
226
227      client.loop();
228    }
229
230    unsigned long now = millis();
231
232    if (now - lastTelemetryTime >= telemetryInterval) {
233      lastTelemetryTime = now;
234
235      float t = dht.readTemperature();
236      float h = dht.readHumidity();
237
238      if (!isnan(t) && !isnan(h)) {
239        sensorValid = true;
240        suhuTerakhir = t;
241
242        processSafety(t);
243
244        StaticJsonDocument<512> docOut;
245        docOut["node_id"] = clientId;
246        docOut["suhu"] = t;
247        docOut["kelembapan"] = h;
248        docOut["status_led"] = digitalRead(ledPin);
249        docOut["kondisi_suhu"] = t > batasSuhu ? "PANAS" : "NORMAL";
250        docOut["uptime_detik"] = millis() / 1000;
251        docOut["ip_lokal"] = WiFi.localIP().toString();
```

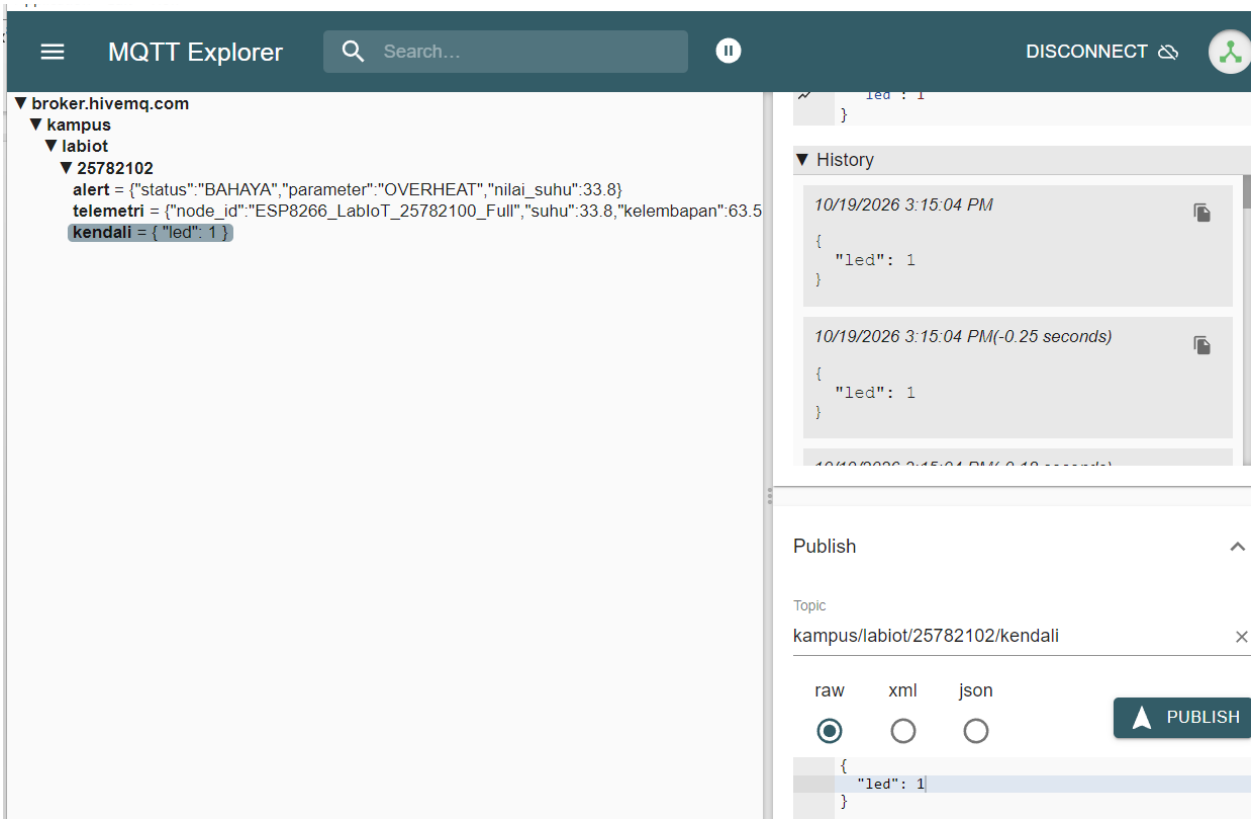
```
244        StaticJsonDocument<512> docOut;
245        docOut["node_id"] = clientId;
246        docOut["suhu"] = t;
247        docOut["kelembapan"] = h;
248        docOut["status_led"] = digitalRead(ledPin);
249        docOut["kondisi_suhu"] = t > batasSuhu ? "PANAS" : "NORMAL";
250        docOut["uptime_detik"] = millis() / 1000;
251        docOut["ip_lokal"] = WiFi.localIP().toString();
252
253        char buffer[512];
254        serializeJson(docOut, buffer);
255
256        if (client.connected()) {
257          bool success = client.publish(topicPublish, buffer);
258
259          if (success) {
260            Serial.printf("[TELEMETRI TERKIRIM] %s\n", buffer);
261          } else {
262            Serial.println("[ERROR] Gagal memublikasikan telemetri!");
263          }
264        }
265      } else {
266        sensorValid = false;
267        Serial.println("[ERROR] Gagal membaca sensor DHT22!");
268      }
269    }
270  }
```

HASIL OUTPUT

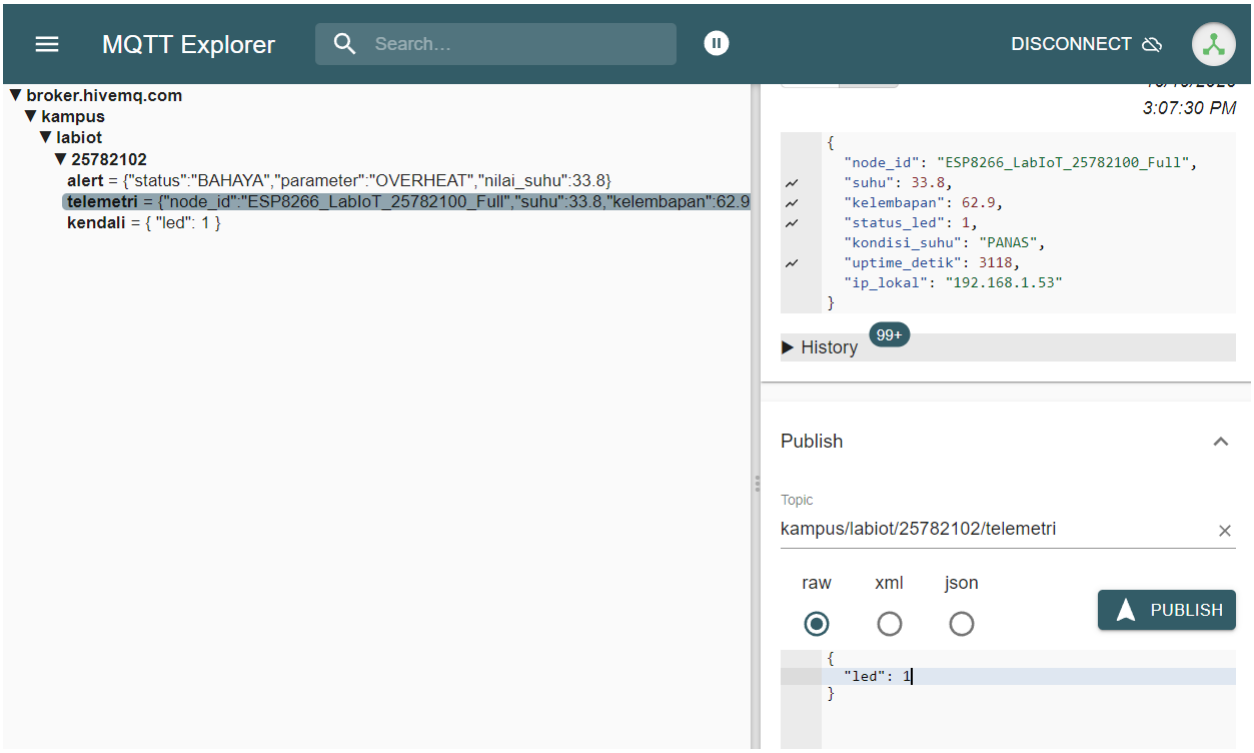


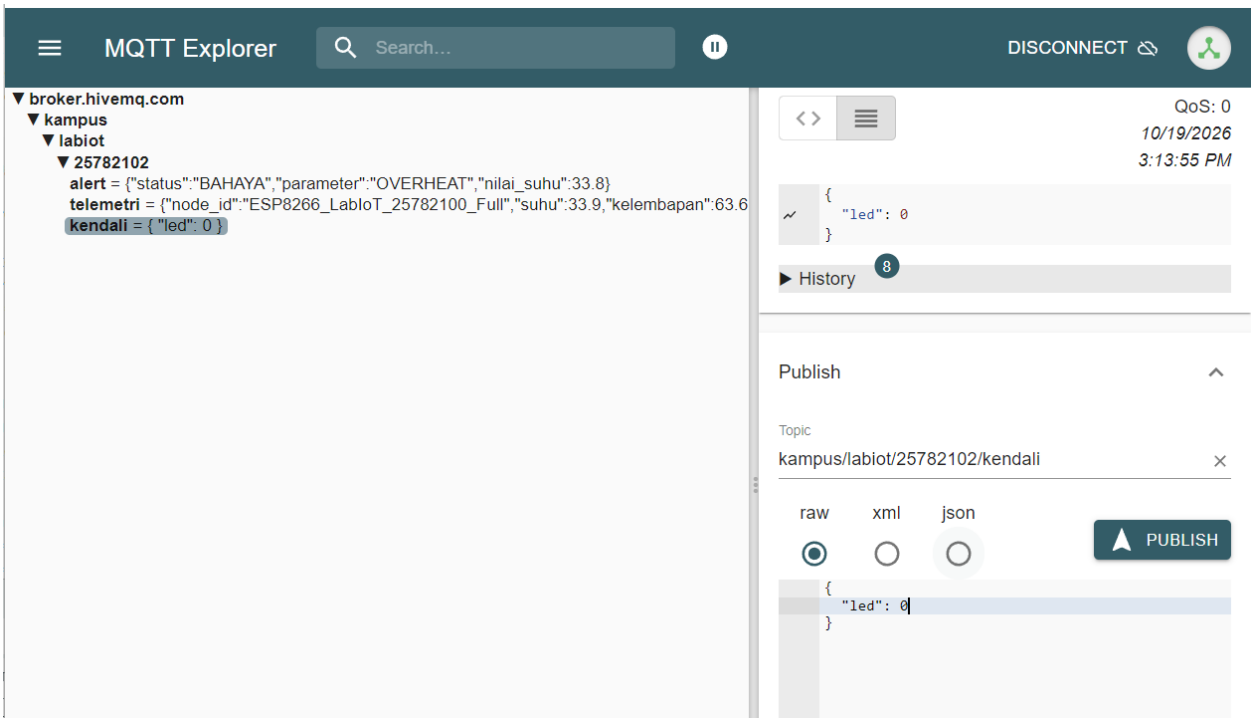
HASIL OUTPUT MQTT

Topic telemetri

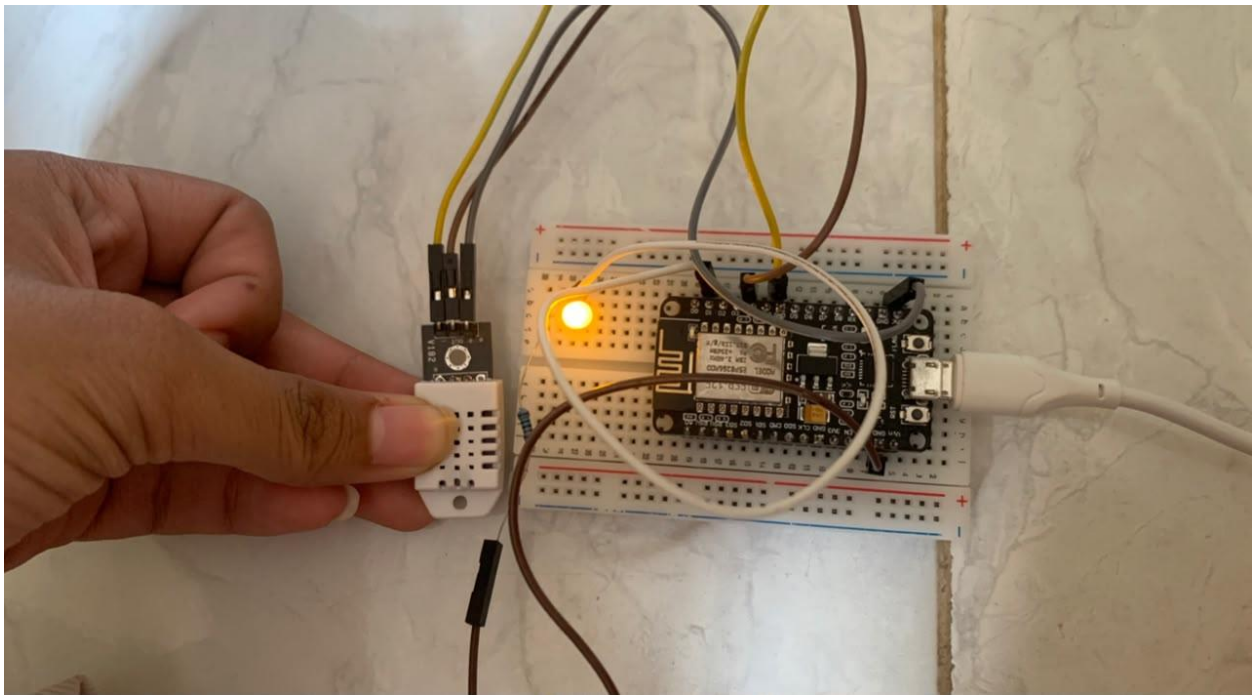


Topic Kendali mengubah menjadi LED 1

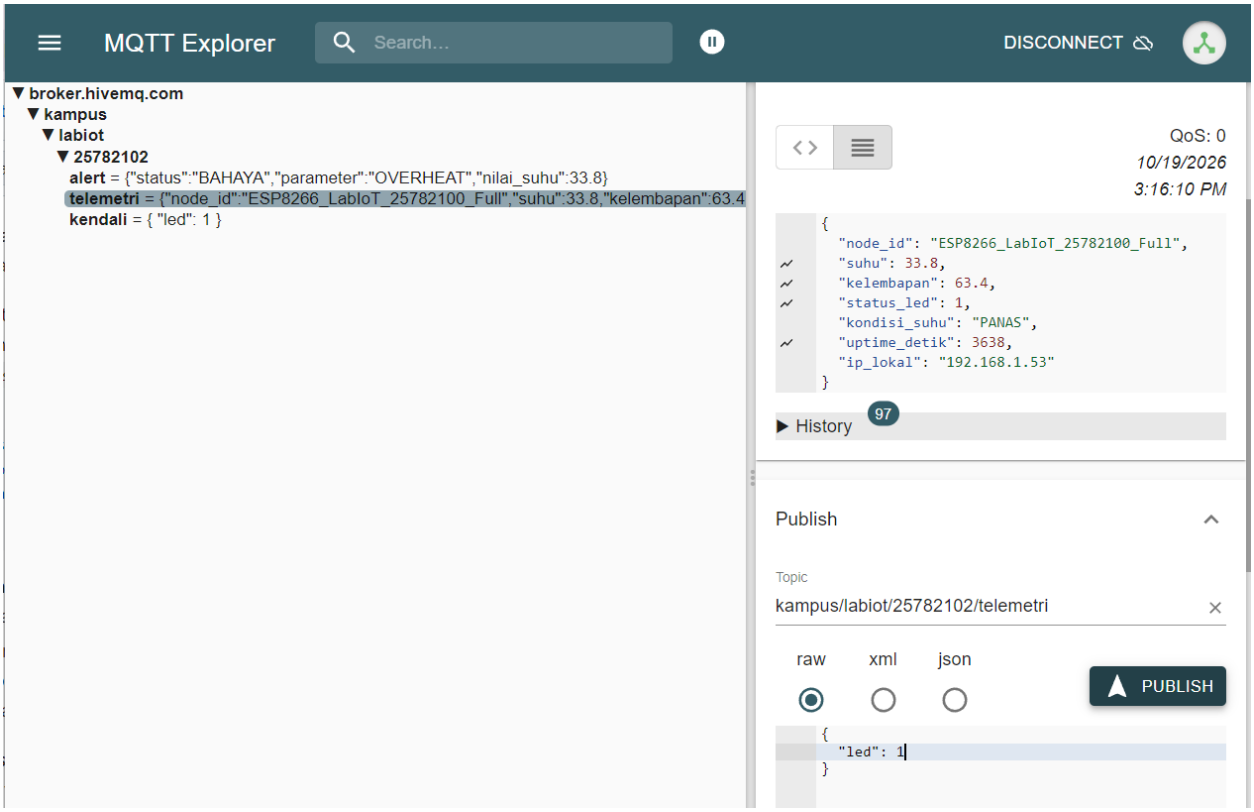




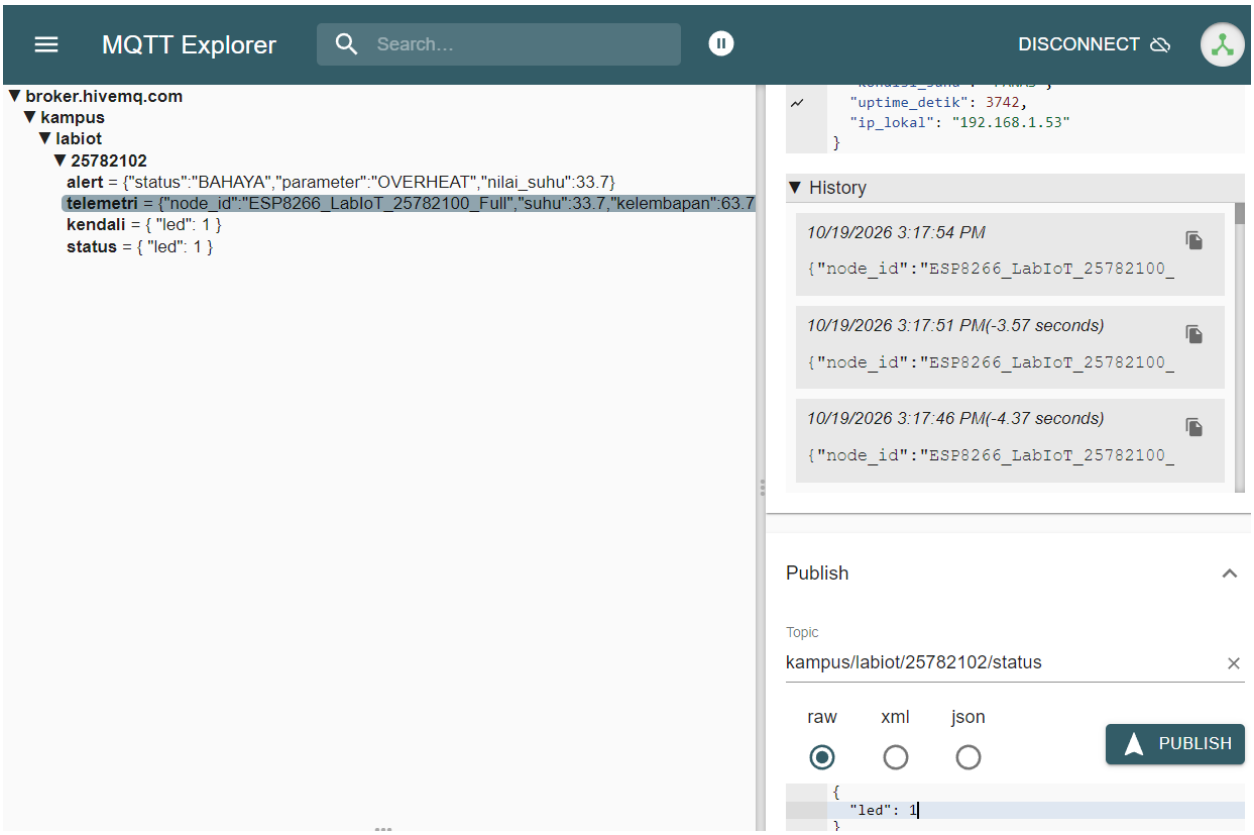
Rangkaian led tetap hidup

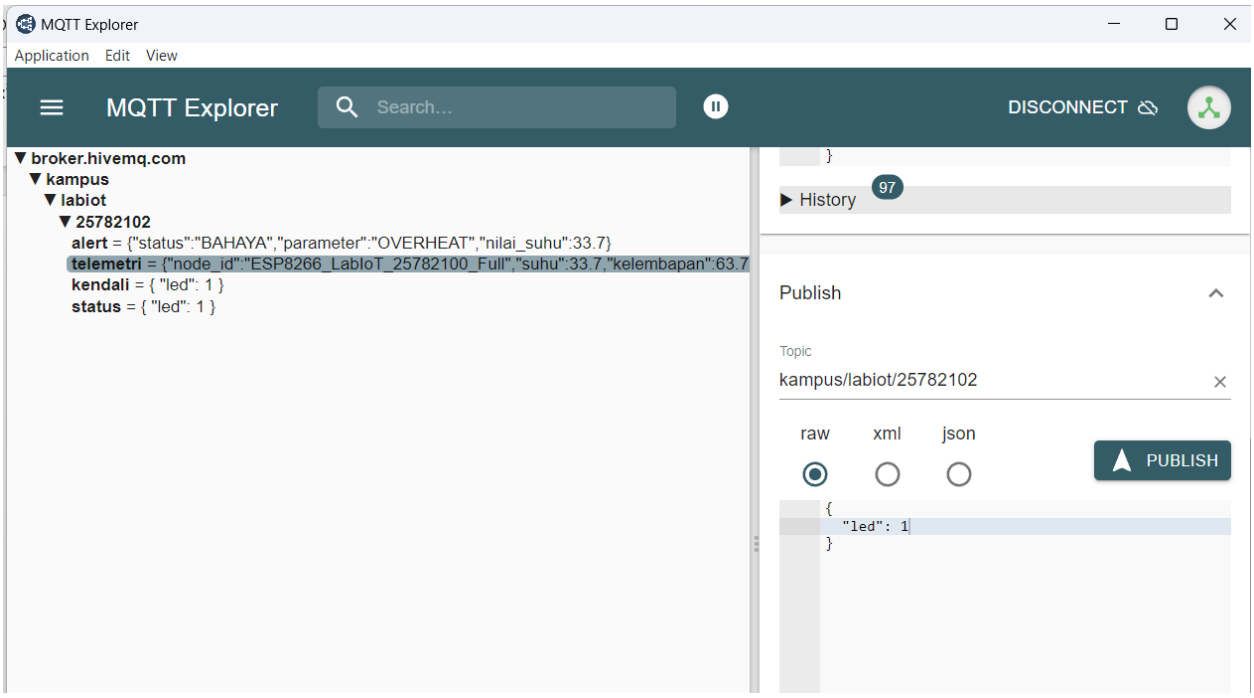


Output topic telemetri



Output MQTT


MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782102
alert = {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":33.7}
telemetri = {"node_id":"ESP8266_LabIoT_25782100_Full","suhu":33.7,"kelembapan":63.7
kendali = { "led": 1 }
status = { "led": 1 }
"uptime_detik": 3742,
"ip_lokal": "192.168.1.53"
History
10/19/2026 3:17:54 PM
{"node_id":"ESP8266_LabIoT_25782100_
10/19/2026 3:17:51 PM(-3.57 seconds)
{"node_id":"ESP8266_LabIoT_25782100_
10/19/2026 3:17:46 PM(-4.37 seconds)
{"node_id":"ESP8266_LabIoT_25782100_
Publish
Topic
kampus/labiot/25782102/status
raw
xml
json
PUBLISH
{
"led": 1


MQTT Explorer
Application Edit View
MQTT Explorer
Search...
DISCONNECT
broker.hivemq.com
kampus
labiot
25782102
alert = {"status":"BAHAYA","parameter":"OVERHEAT","nilai_suhu":33.7}
telemetri = {"node_id":"ESP8266_LabIoT_25782100_Full","suhu":33.7,"kelembapan":63.7
kendali = { "led": 1 }
status = { "led": 1 }
History
97
Publish
Topic
kampus/labiot/25782102
raw
xml
json
PUBLISH
{
"led": 1
}